Firebase Migration Plan

Migration plan: MongoDB → Firebase (Firestore)

Status: **implemented and verified against the Firestore emulator** (register → login → bookmarks → notes → goals/streaks → progress → hifz → settings/email/password-reset, all exercised end-to-end; see the bottom of this doc for what's still needed to go live). This covers replacing MongoDB Atlas + Mongoose with Firebase Firestore (via `firebase-admin`) for all user data (accounts, bookmarks, notes, progress, goals, streaks, hifz). Quran content stays API-sourced either way — unaffected.

The brief explicitly allows this ("MongoDB or your established choice"), so this is not a brief-compliance issue, just an infra swap.

---

0. Scope check — what actually touches Mongo today

26 files import `mongoose`, `connectToDatabase`, or `lib/models/*`: `src/lib/db.ts`, all 7 files in `src/lib/models/`, `src/auth.ts`, `src/lib/bookmarks/favourites.ts`, `src/lib/goals/evaluate.ts`, and 15 route handlers/pages under `src/app/api/account/*`, `src/app/api/auth/reset/*`, `src/app/api/health`, plus the 6 `src/app/account/*/page.tsx` server components that read session/user data.

Auth.js is **not** using a database adapter — it's `session.strategy: "jwt"` with a plain `Credentials` provider that does one `User.findOne()` inside `authorize()` (`src/auth.ts`). That's the single biggest simplification: there's no Auth.js/Firestore adapter to install or configure. We only need to swap what `authorize()` queries.

7 Mongo collections today: `users`, `bookmarks`, `collections` (bookmark folders), `notes`, `progress_events`, `goals`, `streak_states`, `memorised_ayahs`.

---

1. Schema redesign — don't port 1:1, use Firestore idioms

A flat copy of the 7 Mongo collections would fight Firestore's model (no unique compound indexes, no `distinct()`, queries cost per-document-read). Two of Mongo's own workarounds go away for free with a Firestore-native design:

- `Goal`'s `{userId, active}` **partial unique index** existed only to fake "one active goal per user." `evaluate.ts` confirms goal *history* is never read — streak math only ever uses *today's* active goal. So the active goal doesn't need to be its own collection at all; it's just a field on the user doc. Same for `StreakState` (already a per-user singleton in Mongo, `unique: true` on `userId`).

- Compound-unique indexes like `{userId, verseKey}` on `Bookmark`/`Note`/`MemorisedAyah` and `{userId, surah, date}` on `ProgressEvent` all become **free** once each is a subcollection under `users/{userId}` and the natural key becomes the *document ID*. `docRef.create()` in the Admin SDK throws `ALREADY_EXISTS` if the ID is taken — exact parallel to Mongo's E11000 duplicate-key race handling already coded in `bookmarks/route.ts`.

Recommended schema:

```
users/{userId}
  email: string (lowercased)
  passwordHash: string
  profile: { displayName, avatarUrl }
  roles: string[]                    // unchanged, still unused in UI
  moderation: { flagged, suspended }
  settings: map
  lastPosition: { verseKey, surahId, ayahId, updatedAt } | null
  emailVerified: timestamp | null
  passwordResetToken / passwordResetExpires / passwordResetRequestedAt
  activeGoal: { type, target } | null       // was Goal collection
  streak: { currentStreak, longestStreak, lastMetDate }  // was StreakState collection
  viewedSurahs: number[]              // NEW — see §1a
  createdAt / updatedAt

userEmails/{emailLower}
  userId: string                      // uniqueness reservation, see §2

users/{userId}/bookmarks/{verseKey}          // doc ID = "2:255" etc — uniqueness for free
  collectionId: string
  createdAt: timestamp

users/{userId}/bookmarkCollections/{collectionId}   // auto-ID; small (<20/user), fetch-all
is fine
  name: string
  isDefault: boolean
  createdAt / updatedAt

users/{userId}/notes/{verseKey}
  text: string
  createdAt / updatedAt

users/{userId}/memorisedAyahs/{verseKey}
  surahId, ayahId, memorisedAt

users/{userId}/progressEvents/{surah}_{yyyy-mm-dd}   // doc ID encodes the unique key
directly
  surah, fromAyah, toAyah, date, createdAt
```

1a. One real behavior change worth flagging: `viewedSurahIds`

Today, `GET /api/account/progress` calls `ProgressEvent.distinct("surah", { userId })`. Firestore has no `distinct()` — the literal port is "read every progress-event doc the user has ever created and dedupe client-side." That's fine for a light reader but scales badly for a daily user over years (one doc per surah *per day*, not capped at 114). Firestore bills per document read, so this would get silently more expensive over time.

Fix: maintain `users/{userId}.viewedSurahs: number[]` and update it with `FieldValue.arrayUnion(surah)` in the same write that records a progress event. Reading progress becomes a single user-doc read instead of an unbounded subcollection scan. This is a schema improvement over the Mongo version, not just a port.

1b. Bookmark count cap (`MAX_BOOKMARKS = 2000`)

Mongo uses `countDocuments()`. Firestore's `.count()` aggregation query (billed as 1 read per ≤1000 matched docs, not a full document read) is the direct equivalent — cheap, no schema change needed.

1c. Surah-prefix bookmark query

`bookmarks/route.ts` does `verseKey: { $regex: "^2:" }` to fetch one surah's bookmarks for the reader. Firestore equivalent is a range query: `.where('verseKey', '>=', '2:').where('verseKey', '<', '2:')` — works the same way, no composite index needed since it's a single field within an already-scoped subcollection.

---

2. Email uniqueness (the one thing Firestore doesn't give you for free)

Mongo enforced `email: { unique: true }` at the database level. Firestore only guarantees uniqueness of a **document ID**. Since the app supports changing email (`/api/account/settings/email`), using the email itself as `users/{userId}`'s ID is a bad fit — you can't cheaply rename a Firestore document (subcollections don't move with it).

Pattern: a small reservation collection, written inside a transaction:

- **Register:** transaction reads `userEmails/{emailLower}`; if it exists, abort with "email taken"; else create `userEmails/{emailLower} → {userId}` and `users/{userId}` together.
- **Change email:** transaction reads `userEmails/{newEmailLower}`; if taken, abort; else delete `userEmails/{oldEmailLower}`, create `userEmails/{newEmailLower}`, update `users/{userId}.email`.
- **Login:** read `userEmails/{emailLower}` → `userId`, then read `users/{userId}` (2 reads instead of Mongo's 1 `findOne`, negligible cost).

This is the direct Firestore-native replacement for a unique index and needs to be transactional to avoid a race where two registrations for the same email both pass the "doesn't exist" check.

---

3. Password reset token expiry (TTL)

Mongo doesn't auto-expire these either today (checked manually against `passwordResetExpires` in the route) — no change in behavior needed. Optionally add a **Firestore TTL policy** on `passwordResetExpires` at the collection-group level so stale tokens get garbage-collected automatically instead of sitting on the user doc forever. Nice-to-have, not required for parity.

---

4. Infra & package changes

- Remove `mongoose` from `package.json`; add `firebase-admin`.
- Remove `MONGODB_URI` from `.env.example`; add:
  - `FIREBASE_PROJECT_ID`
  - `FIREBASE_CLIENT_EMAIL`

    - `FIREBASE_PRIVATE_KEY` (service account, from Firebase Console → Project Settings → Service Accounts → Generate new private key)
    - These map to `firebase-admin`'s `cert({ projectId, clientEmail, privateKey })` initializer. All account/auth routes already declare `export const runtime = "nodejs"`, which is required — `firebase-admin` doesn't run on the Edge runtime, same constraint Mongoose already had.
- Firestore mode: **Native mode**, not Datastore mode.
- **Security rules:** since every access goes through the server-side Admin SDK (which bypasses rules entirely) and there's no plan to add a client-side Firestore SDK, rules should just be `allow read, write: if false;` — deny-all. This matches the current architecture where the browser never talks to the database directly, only through `/api/account/*`.
- **Composite indexes:** two are actually needed (caught during implementation, not obvious from the schema alone): `bookmarks` filtered by `collectionId` (equality) then sorted by `createdAt` (different field) needs one, and `memorisedAyahs` sorted by `(surahId, ayahId)` needs another. Both are declared in `firestore.indexes.json`. Everything else — single-field filters/sorts, and the surah-prefix range queries on `verseKey`/document-id — stays index-free.
- Firebase CLI (`firebase-tools`) for local emulator + `firestore.rules`/`firestore.indexes.json` deploy — add as a dev dependency, wire `firebase emulators:start` for local dev so `pnpm dev` doesn't require hitting production Firestore.

---

5. Data-access layer rewrite

Replace `src/lib/db.ts` + `src/lib/models/*.ts` with a Firestore equivalent. Suggested shape (names illustrative):

```
src/lib/firestore/
  admin.ts          # cached firebase-admin app + Firestore() singleton (mirrors db.ts's
HMR-safe caching pattern)
  users.ts          # getUserById, getUserByEmail (via userEmails), createUser, updateUser,
changeEmail (transaction)
  bookmarks.ts       # list/create/update/delete, mirrors bookmarks/route.ts's shape
  bookmarkCollections.ts
  notes.ts
  progress.ts        # recordProgressEvent (writes progressEvents doc + arrayUnion
viewedSurahs), sumAyahsForDay
  goals.ts            # embedded on user doc — thin wrapper, no dedicated collection
  hifz.ts             # memorisedAyahs subcollection helpers
```

Mongoose gave free schema validation (`maxlength`, `min`/`max`, `enum`) — Firestore has none. Add a thin `zod` (or hand-rolled) validation layer at the top of each write path so the same input constraints (e.g. note text max length, goal target 1–6236, surah 1–114) are still enforced before writing. This is new code, not a port — Mongoose was quietly doing this for you everywhere.

File-by-file touch list (26 files)

Category	Files	Change
DB connection	`src/lib/db.ts`	Replace with `src/lib/firestore/admin.ts`
Models	`src/lib/models/*.ts` (7 files)	Delete; logic moves into `src/lib/firestore/*.ts` repository functions
Auth	`src/auth.ts`	Swap `User.findOne()` for `getUserByEmail()`
Auth		

Category	Files	Change
	`src/lib/auth/credentials.ts`, `src/lib/auth/session.ts`	Check for Mongo-specific assumptions (ObjectId string format) — session ID becomes a Firestore auto-ID string instead of a 24-hex Mongo ObjectId; anywhere that validates `mongoose.Types.ObjectId.isValid(id)` (bookmarks route, collections route — 5 occurrences) needs a different validity check (Firestore IDs are opaque strings; just validate non-empty/length, ownership is enforced by the query path instead)
Business logic	`src/lib/bookmarks/favourites.ts`, `src/lib/goals/evaluate.ts`	Rewrite against new repository functions; `evaluate.ts`'s goal/streak logic is unchanged, only the storage calls change
Routes	12 files under `src/app/api/account/*` and `src/app/api/auth/reset/*`	Swap Mongoose calls for repository calls; the `mongoose.mongo.MongoServerError` duplicate-key catch in `bookmarks/route.ts` becomes a `docRef.create()` + catch `ALREADY_EXISTS`
Health check	`src/app/api/health/route.ts`	Swap Mongo ping for a trivial Firestore read
Pages	6 files under `src/app/account/*/page.tsx`	Only change if they import models directly for SSR reads — check each; most likely already go through the API routes or `getSessionUserId()`

---

6. Existing production data

The site is live at rememberquran.com and M4 (accounts) has been deployed, so there may be real registered users in MongoDB Atlas right now. Before cutover:

1. **Check first** — `mongosh` or Atlas UI: `db.users.countDocuments()`. If it's zero or only test accounts, skip straight to a clean cutover (no migration script needed, just tell existing users they may need to re-register once — acceptable at this stage, but confirm with you first since it's a user-facing decision, not just a technical one).
2. If there's real data, write a one-off Node script (`scripts/migrate-mongo-to-firestore.mjs`) that:
   - Connects to Mongo read-only, connects to Firestore (batched writes, 500 ops/batch).
   - For each user: create `userEmails/{email}` + `users/{userId}` using the **same Firestore auto-ID** for `userId` as a fresh doc (Mongo ObjectIds aren't reused — every downstream reference, e.g. bookmarks, needs the same remapped ID, so build a Mongo-ObjectId → Firestore-ID map first, then migrate dependent collections using that map).
   - Migrate `bookmarks`, `collections`, `notes`, `progress_events`, `memorised_ayahs` into the corresponding subcollections under the mapped `userId`.
   - Fold `goals` (`active: true` row) and `streak_states` into the user doc's `activeGoal`/`streak` fields.
   - Backfill `viewedSurahs` per user from their `progress_events` (one-time distinct scan — the expensive operation §1a is designed to avoid going forward, but it's fine as a single migration-time cost).
3. Dry-run against a **separate Firebase project** (or emulator) first, diff counts against Mongo, only then run against the production Firebase project.

---

7. Rollout strategy

1. New branch, not `main` directly — this touches auth and every account feature; a bad cutover locks users out.
2. Stand up a Firebase project (or ask you for an existing one + service account credentials — this is the one hard blocker: **I can't provision a Firebase project or generate credentials for you**, that needs to happen in the Firebase Console under your account).
3. Build the new `src/lib/firestore/*` layer alongside the existing Mongo layer (both present, nothing wired yet) — lets typecheck/lint pass incrementally instead of one giant breaking commit.
4. Swap routes over one feature group at a time (auth → bookmarks → notes → progress/goals → hifz), running the app against the Firestore emulator locally after each group, checking the corresponding `/account/*` page still works end-to-end in the browser.
5. Run the migration script (§6) against a staging Firebase project, verify.
6. Deploy behind the existing Vercel preview-deployment flow first (per the brief's "push regularly, QA tests the live domain" instruction, this still needs to land on `main` before final QA — but a preview deploy lets you sanity-check production Firestore against real environment variables before merging).
7. Cut over: merge, set production env vars in Vercel, run migration script against prod Firestore, deploy, verify `/api/health`, spot-check login/bookmark/note/goal flows on the live domain, decommission the Mongo Atlas cluster only after a confirmed burn-in period (few days) — keep it around as a cold rollback path, don't delete immediately.
8. Update `plan.md`/`README.md` tech stack + data source tables (currently say "MongoDB Atlas + Mongoose") once this lands.

---

8. Effort estimate

Phase	Work
Firestore repository layer + validation	New code, ~7 modules, largest single chunk of effort
Route rewrites	12 API routes + `auth.ts` — mechanical once the repository layer exists
Email-uniqueness transaction logic	Small but must be correct — race-prone if rushed
Migration script (if real data exists)	Medium — only needed if step 6.1 finds non-empty collections
Local emulator setup + manual QA pass	Full regression across M4 + M5 hifz, per the brief's regression-testing requirement

Realistically this is a multi-session effort, not a single sitting — recommend treating it as its own mini-milestone with the same "self-test before QA" discipline the brief already asks for elsewhere.

Decisions made

1. No existing Firebase project — nothing to migrate data from or into yet.
2. No real user data in production MongoDB — this was a clean cutover, no migration script was needed (§6 of this plan is now moot).
3. Kept Auth.js exactly as-is (JWT + Credentials provider), swapped only the datastore. Confirmed correct in practice — `authorize()` now calls

`getUserByEmail()` from `src/lib/firestore/users.ts` and nothing else about the session/JWT/middleware layer changed.

What's implemented

- `src/lib/firestore/{admin,users,bookmarks,bookmarkCollections,notes,hifz,progress,goals}.ts` — the full repository layer.
- `src/auth.ts` and all 14 account/auth API routes + 7 `account/*/page.tsx` server components rewritten against it.
- `src/lib/db.ts` and `src/lib/models/*` (the entire Mongoose layer) deleted — nothing in `src/` references Mongo or Mongoose anymore.
- `firebase.json`, `firestore.rules` (deny-all — server-only access via Admin SDK), `firestore.indexes.json` (2 composite indexes).
- `package.json` / `.env.example` updated (`firebase-admin`, `firebase-tools`, `FIREBASE_*` env vars, `pnpm firebase:emulators`).
- Verified end-to-end against the Firestore emulator: registration + duplicate-email rejection, login, bookmarks (create/idempotent-dedupe/surah-prefix filter/move/delete + collection counters), notes, hifz, goal setting + streak evaluation, progress-event recording + `viewedSurahs` denormalization, last-position tracking, display name change, email change (including old-email release and re-registration), password reset (request → email token → consume → single-use enforcement), and all 7 `/account/*` pages rendering.
- `npx tsc --noEmit`, `npx eslint .`, and `npx next build` all pass clean — zero new lint errors (the 17 pre-existing ones are unrelated, in `AudioPlayerContext.tsx` and a few hooks).

What's left before this can go live

1. **You create a real Firebase project** (Console → Add project → enable Firestore in Native mode) and generate a service account key (Project Settings → Service Accounts → Generate new private key) — this is the one step that has to happen in your Firebase Console, I can't do it for you.
2. Paste `FIREBASE_PROJECT_ID` / `FIREBASE_CLIENT_EMAIL` / `FIREBASE_PRIVATE_KEY` into Vercel's production env vars (same names as `.env.example`), remove the old `MONGODB_URI`.
3. Deploy `firestore.rules` and `firestore.indexes.json` to that project: `firebase deploy --only firestore --project <your-project-id>`.
4. Deploy the branch, spot-check `/api/health` and a real register/login/bookmark round trip on the live domain before merging to `main`.
5. Decommission the MongoDB Atlas cluster once you're confident (no rush — nothing in the code touches it anymore, so it's just sitting there costing nothing to leave alone for a rollback window).
6. Update `plan.md` / `README.md` tech stack and data-source tables (still say MongoDB Atlas + Mongoose).